CSD493 · EXTENDED RESEARCH · PART II

Complete Faculty Validation of a Browser-Extension Update-Security Project

Problem recurrence, difficulty, taxonomy, recovery, model & dataset strategy — evidence-based

Team: Bind Pratap Singh (2310110084) · Suhani Deepak Agrawat (2310110717)

Advisor: Dr. Sweta Mishra · **Project:** extdrift · B.Tech CSE, Shiv Nadar IoE

Companion to: Part I — Feasibility & Novelty Study · **Prepared:** September 2026

Scope. This document answers twelve additional validation requirements the faculty raised before implementation: that the problem recurs beyond Cyberhaven; why it is technically hard; a full vulnerability taxonomy; what is recoverable after a malicious update; a precise ML task; an *evidence-based* model choice (not a pre-assumed XGBoost); realistic datasets; honest handling of unknown threats; the static/dynamic/honeytrap question; and a leakage-free evaluation. It was written to test the project, and it concludes with a **narrow-and-reframe** recommendation, not a rubber stamp.

Contents

1 Problem Validation Beyond Cyberhaven
2 What Makes This Difficult — prior-art status
3 Vulnerability Taxonomy (beyond code injection)
4 After a Malicious Update — Detect → Recover
5 Model Selection — evidence-based, not assumed
6 What Exactly Will the Model Predict?
7 Dataset Strategy
8 Unknown / Hidden Threats — classification vs discovery
9 Static, Dynamic & Honeytrap Pages
10 Evaluation Methodology
11 Final Synthesis (10 points)
12 Complete Faculty Validation (12 answers)
13 Additional References

FACT established / vendor-documented **FINDING** from a cited source **ASSUMPTION** reasonable, unverified

OURS our interpretation/design

Headline for the faculty

The problem is **recurring** (a decade of incidents, tens of millions of users), the task is **genuinely hard** (obfuscation, dynamic behaviour, data scarcity — each only partially solved), and the honest project is **narrowed and reframed**, not rejected: *deterministic dynamic behavioural update-diffing, store/enterprise/researcher-positioned, with an experimentally-chosen model.*

1. Problem Validation Beyond Cyberhaven — recurring, not a one-off

FINDING Malicious/compromised/subverted extensions are a **decade-long, repeating** pattern across independent actors and entry vectors:

Incident (year)	What happened	Entry vector	Users	Why defences failed	Diff catch?
DataSpii (2019)	8+ exts exfiltrated history/PII	Permission abuse	~4M	Runtime exfil unmonitored	Partly
Nano Adblocker (2020)	Sold; data collection added	Ownership transfer	~300k	Trust not re-checked after sale	**Yes**
Great Suspender (2021)	Sold; tracking + remote code	Ownership transfer	~2M	Update passed; no behaviour re-check	**Yes**
Palant "SeaSearch" (2023)	Hidden JS-injection API in 34 exts	Bad code in updates/origin	**~87M installs**	Obfuscated, server-triggered; static missed	**Yes** (update variants)
Cyberhaven + campaign (2024)	Cookie/token theft via update	OAuth phishing of devs	**~2.6M / ~35 exts**	Account compromise; update passed review	**Yes**
Chrome+Edge spyware (2025)	18–19 exts spying/hijacking	Fake tools / subverted	"millions"	Long-dormant trusted listings	Partly/Yes
Crypto stealers (2025–26)	Popular exts weaponised for wallet theft	Subverted popular exts	~80k (one)	Targeted, condition-gated	**Yes**

Sources: BleepingComputer (75M), Kaspersky (87M), Sekoia (Cyberhaven), Malwarebytes (2025), Infosecurity (18 exts), chrome-mal-ids.

> **Conclusion** OURS
>
> Two sub-patterns recur independently — **supply-chain/account compromise** (Cyberhaven) and **ownership-transfer / acquire-and-subvert** (Great Suspender, Nano) — and **both express as a behavioural change between a trusted version and its successor**, exactly what we score. Palant's campaigns show the **obfuscation/dormancy** that defeats static review, motivating our dynamic stance. The project is **not built around one incident**.

2. What Makes This Difficult — challenge-by-challenge

Challenge	Prior-art status	Notes & our exposure
Obfuscation / minification	**Partial**	Defeats static (VEX, You've-Changed admit this). Dynamic sidesteps it — **our advantage**.
Dynamic / condition-gated behaviour	**Partial**	Hulk elicits via HoneyPages; coverage never complete; server-gated payloads can stay dormant.
Run-to-run non-determinism	**Weakly solved**	Record/replay determinism (ours) is the mitigation — **our core contribution**.
Third-party deps / supply chain	Partial	Behavioural diff catches the *effect* regardless of source.
Large permission sets / browser APIs	Partial	Declaration ≠ behaviour; MV2/MV3 + `chrome.*` / `browser.*` normalisation needed.
Static vs dynamic trade-off	Known	Each alone insufficient; hybrid is consensus.
False positives / negatives	**Open (the bottleneck)**	2025 "It's not Easy": 98% lab, >1,000 wild flags, only 68 real. Determinism + diff target FPs.
Previously-unknown payloads	**Open**	Supervised classifies *known*; novelty needs anomaly methods (§8).

Challenge	Prior-art status	Notes & our exposure
Runtime depth (MV3 service worker)	Partial	SW exfiltration is a page-centric blind spot; we instrument it.
Scalability	Solved (engineering)	Hulk did ~48k via parallel automation; not a research blocker.
Dataset scarcity (update pairs)	**Open**	No public labelled update-pair set → synthetic-train + real-validate (§7).

> *The concentration of **partial/open** rows — determinism, false positives, unknown payloads, update-pair data — is what makes this a **research** project, not an implementation task. We attack **determinism + false positives on the update-diff**, the open cell nearest our closest prior work.*

3. Vulnerability Taxonomy (beyond code injection)

OURS synthesised from the literature. **S**=static-detectable, **D**=needs dynamic, **ML?**=learning realistically helps on our features. ✓ yes · ~ partial · ✗ no.

Vulnerability class	How it occurs	Impact	S	D	ML?	Relevance
1 Malicious update / supply-chain	Trusted ext updated with bad code	Mass theft	~	✓	✓	**Core**
2 Credential / cookie / token theft	Read login fields / `cookies.getAll` → exfil	Account takeover	~	✓	✓	**Core**
3 Data exfiltration	Read PII → outbound to new host	Privacy breach	~	✓	✓	**Core**
4 Excessive / escalating permissions	Update adds `<all_urls>`, `cookies`	Wider capability	✓	~	✓	Static signal
5 MV3 service-worker abuse	Background beaconing, no page	Stealth exfil	✗	✓	✓	**Core**
6 Browser-API misuse	`webRequest` / `DNR` / `scripting`	Tamper, inject	~	✓	✓	Core-adjacent
7 DOM injection & UXSS	Inject nodes / bypass isolation	Page hijack, XSS	~	✓	~	Partial
8 Insecure / cross-extension messaging	Unvalidated `onMessage`, `externally_connectable`	Privilege bridge	✓	~	~	Static signal
9 Origin-validation failure	Content script trusts page data	Escalation	✓	~	✗	Secondary
10 Insecure native messaging	Talks to native host	Sandbox escape	✓	~	~	Out of core
11 CSP / policy weakening	Remote code (MV2) / rules channel (MV3)	RCE-like	✓	~	~	Manifest diff
12 Affiliate / ad / search hijack	DNR redirects, script inject	Fraud, tamper	~	✓	✓	Behavioural
13 Fingerprinting / privacy	Probe environment, track	Deanonymise	~	✓	~	Secondary
14 Logic flaws / vulnerable benign (VEX)	Exploitable flow in benign code	Web-triggered escalation	✓	~	✗	**Out of scope**

> *extdrift centres on classes **1–6 and 12** — the behavioural/capability classes that are the actual payloads in §1 and are best seen dynamically. Classes 8–11 and 14 (static, code-vulnerability flavoured) are **explicitly out of scope** — conflating them is the over-broad "vulnerability detection" objective to avoid (§6).*

4. After a Malicious Update — Detect → Block → Disable → Rollback → Recover

FACT Five different capabilities, held by different parties:

Stage	Chrome/Chromium	Firefox	End user?	Notes
Detect	Google review/telemetry; Safety Check	AMO review + blocklist	No (limited)	Mostly vendor-side, often **after** exposure
Block (new installs)	CWS takedown + blocklist	AMO blocklist	No	Stops further spread, not those already updated
Disable (installed)	**Remote kill-switch** (malware flag); enterprise blocklist	Blocklist auto-disables	Partial	Malware-flagged ext can't be re-enabled by user
Rollback (prev version)	**Not for users**; enterprise version-pin	No user rollback	**No**	Stores hold all versions & *could* revert; users cannot
Recover (undo damage)	Out of browser scope	—	**No**	Stolen tokens already out → rotate creds, revoke sessions

Blunt conclusions

Prevention/rollback for an individual user is essentially unavailable — updates are silent, there is no "revert extension" button, and by the time a malicious update runs, **exfiltration may already be complete** (irreversible at the browser layer; only service-side credential rotation helps). Disable/block exist but are **vendor/enterprise-controlled and usually post-exposure**. **This is exactly why our value is early detection at the store/enterprise/researcher tier — before propagation — not consumer recovery.**

5. Model Selection — evidence-based, not "XGBoost because everyone uses it"

FINDING Published extension detectors are dominated by **classic tabular ML on static/metadata features** (SVM, RF, KNN, XGBoost; combined ML+feature-engineering >95% lab — Springer 2023). On tabular data, gradient-boosted trees usually edge out RF/SVM (Grinsztajn, NeurIPS 2022) — so XGBoost is a **reasonable prior, not a proven winner for us**.

Model	Fit for Δ (tabular, small)	Interpretability	Cost	Verdict for us
Logistic Regression	Weak–moderate	High	Trivial	**Floor baseline**
Weighted rules (ours)	Good (hand-tuned)	**Highest**	None	**Control** — ML must beat this
Random Forest	**Strong**	High	Low	**Baseline**
XGBoost / LightGBM	**Strong (usually best)**	Med (+SHAP)	Low–med	**Leading candidate — prove it**
SVM (RBF)	Moderate	Low	Med	Secondary baseline
MLP / deep nets	Weak on small tabular	Low	High	**Reject for Tier 1**
Sequence (Transformer/LSTM)	Needs event sequence	Low	High	**Tier 2**

Model	Fit for Δ (tabular, small)	Interpretability	Cost	Verdict for us
Graph (GNN)	Needs data-flow graph	Low	High	**Tier 2 (novelty)**
Anomaly (OC-SVM/IsoForest/AE)	Good for *unknown*	Med	Low–med	**Complementary** (§8)

> **The defensible procedure (this is the answer)** OURS
>
> We do **not** pre-declare a winner. We run a **bake-off** — LogReg (floor), rules (control), RF, XGBoost, SVM — on the **same Δ features**, and **select by PR-AUC and false-positive-rate at fixed recall**, under **time-based, extension-level cross-validation** (§10), reporting accuracy, precision, recall, F1, ROC-AUC, PR-AUC and the confusion matrix, with class imbalance handled and **SHAP** for explanation. The final model is whatever wins — our prior is XGBoost, but it must earn it.

6. What Exactly Will the Model Predict?

```
INPUT     : two versions (v_N trusted baseline, v_{N+1} candidate) of the SAME extension
FEATURES  : Δ = f(T_{N+1}) − f(T_N)  − the 24-dim behavioural-difference vector
            (+ static manifest / permission delta)
MODEL     : the bake-off winner (§5)  →  probability
PREDICTION: P(the update introduced malicious / security-relevant behavioural change) ∈ [0,1]
ACTION    : rank / flag the update for review + explainable evidence (fired rules + SHAP)
```

OURS **What it is not, and cannot establish (state plainly):** it is **not** "is this extension malicious" (single-version), **not** code-vulnerability discovery, **not** zero-day vulnerability finding. It **cannot** prove intent/legality; cannot see payloads that **never trigger** in our run (server-gated/time-bombed); cannot judge behaviours **absent from training** without the anomaly layer (§8); cannot generalise across unseen extension *families* without anti-memorisation care (§10). Output is **decision support**, not an automated verdict.

7. Dataset Strategy — what exists, why we must build our own

Source	Samples	Labels	Versions?	Limits
mallorybowes/chrome-mal-ids	1000s of malicious IDs/IOCs	malicious	No	IDs only, no code/versions
"It's not Easy" ground truth (2025)	341 malicious	malicious	No	Small; single-version
Antivirus-for-extensions set	990 malware / 22 benign *(verify)*	both	No	Tiny benign side; single-version
You've Changed corpus	~922k versions / 6 yr	weak (rating-anomaly)	**Yes**	Static-oriented; weak labels; not a clean download
chrome-stats / crx4chrome	version histories, old .crx	none	**Yes**	Unofficial; integrity caveats; popular only
GitHub open-source extensions	many	benign	**Yes (tags)**	**Best real benign update-pair source**
Web Store crawl	~100k+	none	latest only	No history; unlabeled

No public labelled dataset of malicious update pairs exists OURS

So we **construct our own**: (1) **real benign pairs** — GitHub releases + archive versions + weekly self-snapshots → the false-positive set; (2) **synthetic malicious pairs** — real benign v1 + injected payload families → the detection set; (3) **real anchors** — reconstructed Cyberhaven + archivable IDs from chrome-mal-ids → held-out external validity. This is the honest **synthetic-train / real-validate** design, forced by the evidence.

8. Unknown / Hidden Threats — classification vs discovery

OURS Keep two claims strictly separate: **(A)** classification of *known* malicious behaviour patterns (supervised on Δ) — realistic, our main mode; **(B)** discovery of previously-unknown *vulnerabilities* (zero-days) — **we do NOT claim this**; it is a different research problem.

FINDING For *unseen malicious updates* (payloads not in training), anomaly/novelty methods learn "normal benign-update behaviour" and flag deviation — **One-Class SVM, Isolation Forest, autoencoders** (Deep Learning for Zero-day Malware, ACM CSUR 2023). Because our baseline is the extension's **own prior version**, "normal" is unusually well-defined, which *helps* novelty detection.

- OURS **Realistic:** flag an update whose Δ is **anomalous relative to benign updates** even if the exact payload was never seen — a complementary anomaly layer on Δ.
- OURS **Do not claim:** "our ML finds zero-day vulnerabilities." We detect novel malicious *behaviour change*, not novel code vulnerabilities, and only for behaviours our scenarios **trigger**.

9. Static, Dynamic & Honeytrap Pages — what we actually need

- **Static analysis** — keep a **thin** layer only (manifest/permission diff): cheap, pre-execution. **Not** the core; obfuscation/WASM/dynamic code defeat it.
- **Dynamic execution + instrumentation + runtime monitoring — required.** The only way to see obfuscated/condition-gated payloads and MV3 service-worker exfiltration. Already implemented.
- **Honeytrap / deception pages (Hulk HoneyPages) — partially adopt.** They elicit dormant behaviour by satisfying any selector an extension probes — powerful, but **adaptive pages are non-deterministic**, which fights record/replay. OURS We use **scripted deterministic scenarios + a fixed deception test-site** (fake bank/webmail/ collector) — a lighter, determinism-friendly descendant. Full adaptive HoneyPages = stretch.
- **Taint / data-flow** — stretch (Tier-2 graph); our temporal-adjacency `exfil_flows` is the lightweight stand-in.
- **Hybrid static+dynamic** — the recommended architecture.

> *Verdict: dynamic analysis is **necessary, not optional**; deception content is useful and already present in lightweight form; **full adaptive HoneyPages and full taint are scope-inflating** and belong in the stretch tier.*

10. Evaluation Methodology — defensible, leakage-free

- **Extension-level splitting, never file-level.** All versions/pairs of one extension go entirely into train or test — else the model memorises an extension's style and inflates scores. *The single most important anti-leakage rule.*

- **Time-based split** — train on older pairs, test on newer → real deployment, and exposes **concept drift** (Rosenzweig, TWeb 2025).
- **Class imbalance** — report **PR-AUC** and **FPR at fixed recall**; class weights / stratified CV; never accuracy alone.
- **Zero-day/unknown eval** — hold out entire **payload families**; test whether the anomaly layer still flags them.
- **Cross-extension & cross-browser generalisation** — test on unseen extensions (and, stretch, Firefox).
- **Anti-memorisation** — strip library/vendor identifiers; ensure the model keys on **behavioural change**, not on which library/extension it is.
- **Ground truth** — the fake collector's log confirms what actually exfiltrated, so detection is measured against reality, not only labels.

11. Final Synthesis

1. **Problem validation** — a decade of independent incidents, tens of millions of users, two recurring vectors, both expressed as behavioural change on update.
2. **Technical difficulty** — determinism, false positives, unknown payloads, labelled update-pair data are partially/not solved → genuine research.
3. **Vulnerability taxonomy** — 14 classes; we own the behavioural/capability ones, exclude the static code-vulnerability ones.
4. **End-to-end position** — Detect (late), Block/Disable (vendor/enterprise), **Rollback (not for users)**, Recover (irreversible; service-side only) → value = early detection before propagation.
5. **ML strategy** — predict P(update introduced malicious behavioural change) on Δ; decision-support; clear "cannot" boundaries.
6. **Model selection** — bake-off on Δ, chosen by PR-AUC + FPR under time/extension-level CV; XGBoost is prior, not assumption; seq/graph = Tier-2; anomaly = novelty.
7. **Dataset strategy** — construct our own (real benign + synthetic malicious + real anchors); no public update-pair set exists.
8. **Unknown-threat detection** — anomaly on Δ flags novel malicious *behaviour change*; not zero-day *vulnerability* discovery.
9. **Recommended architecture — hybrid:** thin static + dynamic capture (deterministic scenarios + deception site) + update diff + dual scoring (rules + bake-off ML on Δ) + optional anomaly layer. Not full HoneyPages/taint/GNN in Project-1.
0. **Final recommendation — NARROW + REFRAME, do not reject.** Keep dynamic behavioural update-diffing + determinism + explainability. Drop language-aware analysis, consumer prevention, zero-day vulnerability discovery. Reposition for store/enterprise/researcher. Cite You've Changed (CCS 2020) as the static predecessor we improve on dynamically.

12. Complete Faculty Validation — 12 direct answers

1. **Why important?** Trusted, auto-updating, high-privilege code can turn malicious after every existing check, silently, to millions — repeatedly.
2. **Evidence beyond Cyberhaven?** DataSpii, Nano, Great Suspender, Palant's ~87M-install campaign, 2025 Chrome/Edge spyware, 2025–26 crypto stealers (§1).
3. **Why difficult?** Obfuscation defeats static; dynamic behaviour must be elicited and separated from web noise; update-pair data is scarce; false positives are the real bottleneck (§2).

4. **Vulnerabilities beyond code injection?** Malicious updates, credential/cookie/token theft, exfiltration, permission/API abuse, MV3 SW abuse, hijacking, message-passing/native-messaging flaws, CSP weakening, privacy/fingerprinting (§3).
5. **What happens when a malicious update reaches a user?** It usually executes and exfiltrates before detection; browser-layer damage is irreversible (§4).
6. **Detected/blocked/disabled/reverted?** Detect/block/disable: yes, but vendor/enterprise-side and often late. **Rollback: not for users.** Recovery: service-side credential rotation (§4).
7. **Why ML?** To learn feature combinations/weightings the rules can't tune, and give a calibrated rankable score — **only if it beats the rule baseline** (§5).
8. **Why XGBoost — or another?** XGBoost is the tabular prior, not foregone; settled by a bake-off on Δ measured on PR-AUC/FPR (§5).
9. **What data?** A self-constructed corpus — real benign pairs + synthetic malicious + real anchors — because no public labelled update-pair set exists (§7).
10. **Unseen threats?** An anomaly layer on Δ flags novel malicious behaviour change; we do not claim zero-day vulnerability discovery (§8).
11. **Static / dynamic / honeytrap roles?** Dynamic is the core; static a thin cheap layer; deception a lightweight deterministic test-site; full HoneyPages/taint are stretch (§9).
12. **Strongest defensible version?** Deterministic dynamic behavioural update-diffing, hybrid static+dynamic, dual-scored (rules + experimentally-chosen ML on Δ) with an anomaly layer, explainable, store/enterprise/researcher-positioned (§11).

13. Additional References (Part II)

Malicious Chrome extensions, 75M installs removed. *BleepingComputer*. link

Dangerous Chrome extensions, 87M downloads. *Kaspersky*. link

Millions spied on by malicious Chrome/Edge extensions (2025). *Malwarebytes*. link

18 Malicious Chrome/Edge extensions. *Infosecurity Magazine*. link

mallorybowes/chrome-mal-ids — malicious extension IOC list. *GitHub*. link

Detecting Malicious Browser Extensions by Combining ML and Feature Engineering. *Springer 2023*. link

Deep Learning for Zero-day Malware Detection and Classification: A Survey. *ACM Computing Surveys 2023*. DOI

Grinsztajn et al. Why do tree-based models still outperform deep learning on tabular data? *NeurIPS 2022*. arXiv

Rosenzweig et al. Concept drift in extension detection. *TWeb 2025*. PDF

Towards Browser Controls to Protect Cookies from Malicious Extensions (CREAM). *arXiv 2405.06830*. PDF

Chrome enterprise extension management (blocklist / ExtensionInstallForcelist). *Tech Lockdown*. link

> Part I references (You've Changed CCS 2020, ExtPrivA, Hulk, VEX, MV3 Unveiled, Palant, Sekoia, Arcanum, etc.) are listed in the Part I report and not repeated here.

Prepared for the CSD493 progress meeting · extdrift · Part II · Written to test, not assume, the project's validity. Tag key: FACT = established/vendor-documented · FINDING = from a cited source · ASSUMPTION = reasonable but unverified · OURS = our interpretation.